

Linux IPv6 Recon Utility Toolkit - Documentation

Overview

A self-contained Python program that performs actual IPv6 reconnaissance on Linux systems by reading system files and executing network commands. No external dependencies required beyond Python standard library.

System Requirements

- Linux operating system
 - Python 3.6 or higher
 - Root privileges recommended (for reading `/proc/net/if_inet6` and executing `ip` commands)
 - `ip` command available (iproute2 package)
-

Constants

Constant	Value	Description
IPv6_ADDR_GLOBAL	0x01	Global scope address
IPv6_ADDR_LINKLOCAL	0x02	Link-local scope address
IPv6_ADDR_SITELOCAL	0x04	Site-local scope address

IPV6_ADDR_LOOPBACK	0x08	Loopback address
IPV6_ADDR_UNICAST	0x10	Unicast address type
IPV6_ADDR_MULTICAST	0x20	Multicast address type
IPV6_ADDR_6TO4	0x40	6to4 transition address
IPV6_ADDR_UNSPECIFIED	0x80	Unspecified address

Part A: IPv6 Reconnaissance & Intelligence

Address Classification Functions

in6_getAddrType(addr: str) -> int

Determines the type and scope of an IPv6 address.

Parameters:

- `addr`: IPv6 address string

Returns: Bitmask combining address type and scope flags

in6_getscope(addr: str) -> int

Returns the scope of an IPv6 address.

Returns: One of `IPV6_ADDR_GLOBAL`, `IPV6_ADDR_LINKLOCAL`, `IPV6_ADDR_SITELOCAL`, `IPV6_ADDR_LOOPBACK`, or `-1`

Scope & Type Detection Functions

Function	Description	Prefix
<code>in6_islladdr(addr)</code>	Link-local unicast detection	<code>fe80::/10</code>
<code>in6_issladdr(addr)</code>	Site-local unicast detection (deprecated)	<code>fec0::/10</code>
<code>in6_isuladdr(addr)</code>	Unique Local Address detection	<code>fc00::/7</code>
<code>in6_isgladdr(addr)</code>	Global unicast detection	<code>2000::/3</code>
<code>in6_ismaddr(addr)</code>	Multicast detection	<code>ff00::/8</code>
<code>in6_isdocaddr(addr)</code>	Documentation prefix detection	<code>2001:db8::/32</code>

Multicast Type Detection

Function	Description
<code>in6_ismnladdr(addr)</code>	Node-local multicast (<code>ff01::/16</code>)

<code>in6_ismgladdr(addr)</code>	Global multicast (ff0e::/16)
<code>in6_ismladdr(addr)</code>	Link-local multicast (ff02::/16)
<code>in6_ismsladdr(addr)</code>	Site-local multicast (ff05::/16)
<code>in6_isllsnmaddr(addr)</code>	Solicited-node multicast (ff02::1:ff00:0/104)
<code>in6_isaddrllallnodes(addr)</code>	All-nodes multicast (ff02::1)
<code>in6_isaddrllallservers(addr)</code>	All-servers multicast (ff02::2)

Transition Technology Functions

`in6_isaddr6to4(addr: str) -> bool`

Checks if address is 6to4 (2002::/16).

`in6_isaddrTeredo(addr: str, teredo_prefix: str = "2001::") -> bool`

Checks if address is Teredo under the specified prefix.

`in6_6to4ExtractAddr(addr: str) -> Optional[str]`

Extracts IPv4 address from 6to4 address.

`teredoAddrExtractInfo(addr: str) -> Tuple[str, int, str, int]`

Extracts Teredo server, flag, mapped address, and mapped port.

Returns: Tuple of (server_address, flag, mapped_address, mapped_port)

MAC & Vendor Functions

`in6_iseui64(addr: str) -> bool`

Checks if address uses modified EUI-64 interface identifier format.

`in6_addrtomac(addr: str) -> Optional[str]`

Extracts MAC address from an IPv6 address.

`in6_addrtovendor(addr: str) -> Optional[str]`

Identifies vendor from embedded MAC address.

`in6_isanycast(addr: str) -> bool`

Checks if address is a subnet-router anycast address (RFC 2526).

`in6_isincluded(addr: str, prefix: str, plen: int) -> bool`

Tests if an address belongs to a specified prefix.

Part B: IPv6 Identity Operations

`in6_mactofaceid(mac: str, ulbit: Optional[int] = None) -> str`

Converts MAC address to modified EUI-64 format.

Parameters:

- `mac`: MAC address string (format: "xx:xx:xx:xx:xx:xx")
- `ulbit`: Optional U/L bit value (0 or 1)

Returns: EUI-64 interface identifier string

Raises: `ValueError` for invalid MAC format

```
in6_ifaceidtomac(ifaceid_s: str) -> Optional[str]
```

Extracts MAC address from EUI-64 interface identifier.

```
in6_getRandomizedIfaceId(ifaceid: str, previous: Optional[str] = None)
-> Tuple[str, str]
```

Implements RFC 3041 privacy extensions for randomized interface identifiers.

Returns: Tuple of (randomized_interface_id, history_value)

Part C: Network/Topology Modeling

```
in6_cidr2mask(m: int) -> bytes
```

Converts CIDR prefix length to 16-byte binary mask.

```
in6_mask2cidr(m: bytes) -> int
```

Converts binary mask to CIDR prefix length.

```
in6_get_common_plen(a: str, b: str) -> int
```

Calculates common prefix length of two addresses.

```
in6_getnsma(addr: bytes) -> bytes
```

Computes solicited-node multicast address in network format.

```
in6_getnsmac(addr: bytes) -> str
```

Computes multicast MAC address (format: "33:33:xx:xx:xx:xx").

```
in6_getha(prefix: str) -> str
```

Computes home agent anycast address for a /64 subnet.

```
in6_getLinkScopedMcastAddr(addr: str, grp_id: Optional[Union[bytes,
str, int]] = None, scope: int = 2) -> Optional[str]
```

Generates RFC 4489 link-scoped multicast address.

Bitwise Operations

Function	Description
<code>in6_or(a1, a2)</code>	Bitwise OR of two addresses in network format
<code>in6_and(a1, a2)</code>	Bitwise AND of two addresses in network format
<code>in6_xor(a1, a2)</code>	Bitwise XOR of two addresses in network format

`in6_ptop(addr: str) -> str`

Normalizes IPv6 address to canonical compressed form.

`in6_isvalid(addr: str) -> bool`

Validates IPv6 address format.

Net6 Class

IPv6 network object with CIDR notation support.

Constructor:

```
Net6(network: str)  # Example: Net6("2001:db8::/64")
```

Methods:

Method	Description
--------	-------------

<code>__contains__(addr)</code>	Checks if address is in network
<code>__iter__()</code>	Iterates over all addresses in network
<code>__len__()</code>	Returns number of addresses

Properties:

- `network`: Prefix string
- `prefix_len`: Prefix length
- `size`: Number of addresses in network

Part D: System Reconnaissance

SystemRecon Class

Performs actual system reconnaissance on Linux.

`get_interfaces()` -> `Dict[str, Dict[str, Any]]`

Reads `/proc/net/ipv6` to get all IPv6 addresses on all interfaces.

Returns: Dictionary with interface names as keys, containing:

- `addresses`: List of address dictionaries
- `mac`: MAC address string
- `flags`: Interface flags
- `scope`: Scope value

`get_routing_table()` -> `List[Dict[str, Any]]`

Reads `/proc/net/ipv6_route` to get IPv6 routing table.

Returns: List of route dictionaries containing:

- `destination`: Destination network
 - `prefix_len`: Prefix length
 - `next_hop`: Next hop address
 - `metric`: Route metric
 - `interface`: Interface name
 - `network`: Network in CIDR notation
-

`get_neighbors() -> List[Dict[str, Any]]`

Executes `ip -6 neigh show` to get neighbor cache.

Returns: List of neighbor dictionaries containing:

- `address`: IPv6 address
 - `interface`: Interface name
 - `state`: Neighbor state (REACHABLE, STALE, etc.)
-

`get_interface_info(iface: str) -> Dict[str, Any]`

Gets detailed information about a specific interface.

Returns: Dictionary containing:

- `name`: Interface name
 - `mac`: MAC address
 - `addresses`: List of IPv6 addresses
 - `mtu`: MTU value
 - `state`: Interface state (up/down)
-

`ping6(addr: str, count: int = 1, timeout: int = 2) -> bool`

Pings an IPv6 address to check reachability.

```
get_all_global_addresses() -> List[str]
```

Returns all global IPv6 addresses configured on the system.

```
get_all_link_local_addresses() -> List[str]
```

Returns all link-local IPv6 addresses configured on the system.

Part E: Integrated Reconnaissance Class

IPv6Recon Class

Comprehensive IPv6 reconnaissance with actual system analysis.

```
__init__()
```

Initializes reconnaissance by gathering:

- Interface information
 - Routing table
 - Neighbor cache
-

```
analyze_system() -> Dict[str, Any]
```

Performs complete system IPv6 analysis.

Returns: Dictionary containing:

- `interfaces`: Interface analysis results
- `routing`: Routing table analysis
- `neighbors`: Neighbor cache analysis
- `global_addresses`: List of global addresses
- `link_local_addresses`: List of link-local addresses
- `transition_technologies`: Detected transition technologies

- `summary`: Summary statistics
-

`analyze_interfaces()` -> `Dict[str, Dict[str, Any]]`

Analyzes all network interfaces with vendor identification.

Returns: Per-interface dictionary containing:

- `name`: Interface name
 - `mac`: MAC address
 - `vendor`: Vendor name
 - `address_count`: Number of IPv6 addresses
 - `addresses`: List of analyzed address dictionaries
-

`analyze_address(addr: str)` -> `Dict[str, Any]`

Comprehensive analysis of a single IPv6 address.

Returns: Dictionary containing:

- `address`: Original address
- `normalized`: Canonical form
- `valid`: Validity flag
- `type`: "unicast", "multicast", "loopback", "unspecified"
- `scope`: Scope constant
- `is_multicast`: Boolean
- `is_unicast`: Boolean
- `is_global`: Boolean
- `is_link_local`: Boolean
- `is_site_local`: Boolean
- `is_unique_local`: Boolean
- `is_loopback`: Boolean
- `is_unspecified`: Boolean
- `is_6to4`: Boolean
- `is_teredo`: Boolean
- `is_documentation`: Boolean
- `is_eui64`: Boolean
- `is_anycast`: Boolean

- `is_solicited_node`: Boolean
 - `mac_address`: Extracted MAC or None
 - `vendor`: Vendor name or None
 - `embedded_ipv4`: IPv4 from 6to4 or None
 - `reachable`: Reachability test result
-

`analyze_routing()` -> Dict[str, Any]

Analyzes routing table.

Returns: Dictionary containing:

- `total_routes`: Number of routes
 - `routes`: List of route dictionaries
 - `connected_networks`: List of directly connected networks
 - `default_routes`: List of default routes
-

`analyze_neighbors()` -> Dict[str, Any]

Analyzes neighbor cache.

Returns: Dictionary containing:

- `total_neighbors`: Number of neighbors
 - `neighbors`: List of neighbor dictionaries
 - `reachable`: List of reachable neighbors
 - `stale`: List of stale neighbors
-

`detect_transition_technologies()` -> Dict[str, Any]

Detects transition technologies on the system.

Returns: Dictionary with keys:

- `6to4`: List of 6to4 addresses with embedded IPv4
- `teredo`: List of Teredo addresses with server and mapping info

- ula: List of Unique Local Addresses
 - isatap: Placeholder for ISATAP detection
-

get_summary() -> Dict[str, Any]

Returns summary statistics of IPv6 configuration.

Returns: Dictionary containing:

- total_interfaces: Number of interfaces
 - total_addresses: Total IPv6 addresses
 - global_addresses: Count of global addresses
 - link_local_addresses: Count of link-local addresses
 - unique_local_addresses: Count of ULA addresses
 - multicast_addresses: Count of multicast addresses
 - total_routes: Number of routes
 - total_neighbors: Number of neighbors
-

generate_source_identities(interface: str = None) -> List[Dict[str, Any]]

Generates possible source identities for adversary simulation.

Parameters:

- interface: Optional specific interface (uses all if None)

Returns: List of identity dictionaries containing:

- type: "hardware_derived" or "randomized_N"
 - interface: Interface name
 - mac: MAC address (for hardware-derived)
 - eui64: EUI-64 identifier
 - address: IPv6 address
-

get_multicast_targets() -> List[Dict[str, Any]]

Generates multicast targets for reconnaissance.

Returns: List of target dictionaries containing:

- `type`: "solicited_node" or "all_nodes"
 - `interface`: Interface name
 - `multicast_address`: IPv6 multicast address
 - `multicast_mac`: Ethernet multicast MAC
 - `source_address`: Source address (for solicited_node)
-

Part F: OUI Database

`_generate_oui_database()` -> Dict[str, str]

Dynamically generates OUI (Organizationally Unique Identifier) database.

Returns: Dictionary mapping OUI prefixes to vendor names.

Supported Vendors:

- Cisco
- 3Com
- Intel
- Dell
- HP
- Apple
- IBM
- Microsoft
- Sony
- Samsung
- Nokia
- Ericsson
- Motorola
- NEC
- Fujitsu
- Hitachi
- Panasonic
- Toshiba
- Sharp
- Asus
- Acer

- Lenovo
 - Realtek
 - Broadcom
 - Qualcomm
 - Marvell
-

Command-Line Execution

Run the program with root privileges to perform full reconnaissance:

```
sudo python3 ipv6_recon.py
```

The program will display:

1. System IPv6 summary statistics
 2. All network interfaces with addresses and vendor information
 3. Routing table information
 4. Neighbor cache
 5. Detected transition technologies
 6. Generated source identities for adversary simulation
 7. Multicast targets
 8. Optional reachability testing for neighbors
-

Files Accessed

File	Purpose
/proc/net/if_inet6	IPv6 addresses on all interfaces
/proc/net/ipv6_route	IPv6 routing table

<code>/sys/class/net/*/address</code>	MAC addresses
<code>/sys/class/net/*/mtu</code>	MTU values
<code>/sys/class/net/*/operstate</code>	Interface state

Commands Executed

Command	Purpose
<code>ip -6 neigh show</code>	Get neighbor cache
<code>ping6 -c N -W T ADDR</code>	Test reachability

Error Handling

- All file operations are wrapped in try-except blocks
 - Missing files or permissions result in empty data structures
 - Invalid IPv6 addresses are handled gracefully
 - Command execution failures are silently ignored with fallbacks
-

Security Notes

- Requires root privileges for full functionality
- Only reads system files, does not modify system state
- Does not send packets except optional ping tests
- No network scanning beyond configured neighbors

Limitations

- Linux-specific (uses `/proc` filesystem)
- Requires `iproute2` package for neighbor cache
- OUI database is limited to common vendors
- ISATAP detection is not fully implemented
- Does not perform active network scanning

=====

POC :

→ Downloads `sudo python3 5IPv6TargetIntelligenceToolkit.py`

=====

=====

IPv6 REAL System Reconnaissance Toolkit

=====

=====

[1] System IPv6 Summary

Total Interfaces: 3
Total Addresses: 3
Global Addresses: 1
Link Local Addresses: 1
Unique Local Addresses: 1
Multicast Addresses: 0
Total Routes: 9
Total Neighbors: 8

[2] Network Interfaces

Interface: wlp2s0
MAC: 28:16:ad:ac:cd:ee
Vendor: UNKNOWN
Address Count: 1
Address: fe80::94f8:760c:156:aaa6
Type: unicast
Scope: 2

Interface: virbr1

MAC: 52:54:00:a6:3b:f0
Vendor: UNKNOWN
Address Count: 1
Address: fd00:122::1
Type: unicast
Scope: 1

Interface: lo
MAC: 00:00:00:00:00:00
Vendor: Cisco
Address Count: 1
Address: ::1
Type: loopback
Scope: 8

[3] Routing Information

Total Routes: 9
Connected Networks: 9
Default Routes: 3

Default Routes:

Interface: lo
Next Hop: ::
Metric: 4294967295
Interface: lo
Next Hop: ::
Metric: 4294967295
Interface: lo
Next Hop: ::
Metric: 4294967295

Connected Networks:

::/0
fd00:122::/64
fe80::/64
::/0
::1/128

[4] Neighbor Cache

Total Neighbors: 8
Reachable: 0
Stale: 0

[5] Transition Technologies

6to4 Addresses: 0
Teredo Addresses: 0
ULA Addresses: 1

[6] Source Identities (Adversary Simulation)

Generated 12 source identities

Type: hardware_derived
Interface: wlp2s0
MAC: 28:16:ad:ac:cd:ee
EUI-64: 2A16:ADFF:FEAC:CDEE
Address: fe80::2A16:ADFF:FEAC:CDEE

Type: randomized_1
Interface: wlp2s0
Address: fe80::8023:7ae6:6f63:80f5

Type: randomized_2
Interface: wlp2s0
Address: fe80::e9f6:28ff:2357:a8ad

[7] Multicast Targets

Generated 2 multicast targets

Type: solicited_node
Interface: wlp2s0
Multicast: ff02::1:ff56:aaa6
MAC: 33:33:ff:56:aa:a6
Source: fe80::94f8:760c:156:aaa6

Type: all_nodes
Interface: wlp2s0
Multicast: ff02::1
MAC: 33:33:00:00:00:01

=====
=====

Reconnaissance Complete